



UNIVERSIDAD  
**CATÓLICA**  
BOLIVIANA

Departamento de Ingeniería y Ciencias Exactas

Carrera de Ingeniería de Sistemas

## **“Red Mundial De Vuelos”**

*Proyecto Análisis de Algoritmos*

**Cordero Martínez Rubén Gabriel**

**Machaca Villena Alejandro**

Cochabamba - Bolivia

Mayo de 2026

## 1. Descripción del Problema

### 1.1 Contexto General

La aviación comercial constituye uno de los sistemas de transporte más complejos e interconectados del planeta. Cada día, miles de vuelos enlazan aeropuertos de todos los continentes formando una red cuya topología determina la accesibilidad de regiones, la resiliencia ante fallos y la eficiencia global del sistema de transporte aéreo.

En este proyecto se trabaja con el dataset de OpenFlights, una base de datos pública que recopila información real de aeropuertos y rutas aéreas a nivel mundial. A partir de estos datos se construye un grafo dirigido y ponderado que modela la red de vuelos, con el objetivo de responder preguntas concretas sobre alcance, conectividad y eficiencia.

### 1.2 Modelado del Grafo

La red de vuelos se representa como un grafo dirigido y ponderado con las siguientes características:

- **Nodos (vértices):** cada nodo representa un aeropuerto identificado por su ID único. Se almacenan también su nombre, ciudad, país, código IATA, código ICAO y coordenadas geográficas (latitud y longitud).
- **Aristas (arcos):** cada arista conecta un aeropuerto de origen con uno de destino y representa una ruta aérea directa operada por una aerolínea. El peso de cada arista es la distancia geodésica en kilómetros calculada a partir de las coordenadas de ambos aeropuertos.
- **Direccionalidad:** las aristas son dirigidas (un vuelo de A a B no implica necesariamente la existencia de un vuelo de B a A).
- **Estructura de almacenamiento:** se utiliza una lista de adyacencia implementada con `unordered_map<int, vector<Arista>>`, lo que permite acceso eficiente  $O(1)$  promedio a los vecinos de cualquier nodo.

Cada arista almacena el ID de origen, el ID de destino, la distancia calculada, el código de la aerolínea y su ID numérico. Esta información es suficiente para resolver todos los retos planteados.

## **2. Objetivos**

### **2.1 Objetivo General**

Modelar la red mundial de vuelos como un grafo dirigido ponderado y aplicar algoritmos clásicos de la teoría de grafos para analizar las propiedades de alcance, conectividad, eficiencia y diseño mínimo de la red.

### **2.2 Objetivos Específicos**

- Construir el grafo a partir de los archivos airports.dat y routes.dat, aplicando una limpieza previa de los datos para garantizar su consistencia.
- Implementar una búsqueda por amplitud (BFS) para calcular el alcance de un aeropuerto con un máximo de 3 escalas.
- Identificar los componentes fuertemente conexos de la red mediante el algoritmo de Kosaraju, determinando grupos aislados y el grupo más grande.
- Calcular el diámetro del grafo (el camino mínimo más largo) aplicando el algoritmo de Dijkstra desde cada nodo.

## **3. Desarrollo**

### **3.1 Limpieza y Preparación de los Datos**

#### **3.1.1 Descripción del Dataset Original**

El dataset de OpenFlights está compuesto por dos archivos de texto separados por comas (CSV):

- airports.dat: contiene 7 697 registros de aeropuertos de todo el mundo, con 14 campos por registro (ID, nombre, ciudad, país, IATA, ICAO, latitud, longitud, altitud, zona horaria, DST, base de datos de zonas horarias, tipo y fuente).
- routes.dat: contiene 67 662 registros de rutas aéreas, con 9 campos por registro (aerolínea, ID de aerolínea, aeropuerto de origen en IATA, ID de origen, aeropuerto de destino en IATA, ID de destino, codeshare, paradas y tipo de aeronave).

Ambos archivos presentan valores nulos representados con el texto literal \N y algunos campos de texto que incluyen comas internas encerradas entre comillas, lo que requirió un manejo especial durante la lectura.

### 3.1.2 Estadísticas del Dataset

| Archivo      | Registros originales | Registros limpios | Reducción |
|--------------|----------------------|-------------------|-----------|
| airports.dat | 7 697                | 6 072             | ~21 %     |
| routes.dat   | 67 662               | 36 389            | ~46 %     |

La reducción en rutas se debe principalmente a la eliminación de rutas con IDs nulos, rutas que conectan un aeropuerto consigo mismo (auto-bucles) y rutas duplicadas (mismo par origen-destino).

### 3.1.3 Proceso de Limpieza con Python (Polars)

La limpieza de los datos se realizó con un script en Python utilizando la librería Polars, aprovechando su eficiencia para el procesamiento de archivos de gran tamaño.

Para el archivo de aeropuertos (airports.dat) se aplicaron los siguientes pasos:

- Lectura del archivo indicando la coma como separador, el texto \N como valor nulo y las comillas como carácter de cierre de campos.
- Asignación de nombres descriptivos a las 14 columnas del archivo.
- Selección únicamente de las 8 columnas relevantes para el proyecto (id, nombre, ciudad, país, IATA, ICAO, Latitud y Longitud).
- Filtrado de registros para conservar únicamente los aeropuertos que cuentan con un código IATA válido (no nulo), eliminando así aeropuertos sin identificación internacional estándar.

```
df_airports = pl.read_csv(  
    "airports.dat",  
    separator="," ,  
    null_values="\N",  
    quote_char="\"",  
    has_header=False,
```

```
    new_columns=["id","nombre","ciudad","pais","IATA","ICAO","Latitud","Lon  
    gitud",
```

```

        "Altitud", "Zona Horaria", "DST", "tz_database", "Tipo", "Fuente"]
    )
    df_airports_final = df_airports.select(["id", "nombre", "ciudad", "pais",
                                           "IATA", "ICAO", "Latitud", "Longitud"])
    .filter(pl.col("IATA").is_not_null())

```

Para el archivo de rutas (routes.dat) se aplicaron los siguientes pasos:

- Lectura con la misma configuración de separador, nulos y comillas que el archivo de aeropuertos.
- Asignación de los 9 nombres de columnas correspondientes.
- Selección de los 4 campos necesarios: ID de destino, ID de origen, código de aerolínea e ID de aerolínea.
- Filtrado para eliminar registros con cualquier campo nulo.
- Eliminación de rutas donde el aeropuerto de origen y destino sean el mismo (auto-bucles sin sentido aeronáutico).
- Cruce con la lista de aeropuertos válidos para descartar rutas que referencien aeropuertos no presentes en el archivo limpio.
- Eliminación de rutas duplicadas (mismo par origen-destino), conservando solo una dirección por par.

```

df_routes_final = (
    df_routes
    .select(["ID_Destino", "ID_Origen", "Aereolinea", "ID_Aereolinea"])
    .filter(
        pl.col("ID_Origen").is_not_null() &
        pl.col("ID_Destino").is_not_null() &
        pl.col("ID_Aereolinea").is_not_null() &
        (pl.col("ID_Origen") != pl.col("ID_Destino")) &
        pl.col("ID_Origen").is_in(airport_ids) &
        pl.col("ID_Destino").is_in(airport_ids)
    )
    .unique(subset=["ID_Origen", "ID_Destino"])
)

```

### 3.1.4 Estructura Final de los Datos

Los datos limpios se exportaron a dos archivos CSV (airports\_final.csv y routes\_final.csv) con los campos estrictamente necesarios:

| Índice | Campo    | Tipo             | Descripción                                          |
|--------|----------|------------------|------------------------------------------------------|
| 0      | id       | Entero           | Identificador único del aeropuerto (clave del grafo) |
| 1      | nombre   | Texto            | Nombre completo del aeropuerto                       |
| 2      | ciudad   | Texto            | Ciudad a la que sirve                                |
| 3      | pais     | Texto            | País donde se ubica                                  |
| 4      | IATA     | Texto (3 letras) | Código internacional del aeropuerto (ej. VVI)        |
| 5      | ICAO     | Texto (4 letras) | Código ICAO del aeropuerto (ej. SLVR)                |
| 6      | Latitud  | Decimal          | Coordenada geográfica (Norte positivo)               |
| 7      | Longitud | Decimal          | Coordenada geográfica (Este positivo)                |

| Índice | Campo         | Tipo   | Descripción                         |
|--------|---------------|--------|-------------------------------------|
| 0      | ID_Destino    | Entero | ID del aeropuerto de destino        |
| 1      | ID_Origen     | Entero | ID del aeropuerto de origen         |
| 2      | Aereolinea    | Texto  | Código IATA/ICAO de la aerolínea    |
| 3      | ID_Aereolinea | Entero | Identificador único de la aerolínea |

## 3.2 Construcción del Grafo en C++

### 3.2.1 Estructuras de Datos Utilizadas

El grafo se implementó en C++ usando tres estructuras de datos globales definidas en grafo.h:

- aeropuertos (unordered\_map<int, Aeropuerto>): mapa que asocia cada ID de aeropuerto con su información completa. Permite acceso en O(1) promedio.
- adyacencia (unordered\_map<int, vector<Arista>>): lista de adyacencia que, dado el ID de un aeropuerto, devuelve la lista de aristas (rutas) que salen de él. Es la estructura central del grafo.
- iata\_a\_id (unordered\_map<string, int>): mapa auxiliar para buscar el ID numérico de un aeropuerto a partir de su código IATA de 3 letras.

### 3.2.2 Cálculo de Distancia Geodésica

Para ponderar las aristas con distancias reales, se implementó la fórmula de Haversine, que calcula la distancia más corta entre dos puntos sobre la superficie de una esfera (la Tierra) a partir de sus coordenadas de latitud y longitud:

```
double calcularDistancia(double lat1, double lon1, double lat2, double lon2) {
    const double R = 6371.0;      // Radio de la Tierra en km
    const double PI = acos(-1.0);
    double dLat = (lat2 - lat1) * PI / 180.0;
    double dLon = (lon2 - lon1) * PI / 180.0;
    lat1 = lat1 * PI / 180.0;
    lat2 = lat2 * PI / 180.0;
    double a = sin(dLat/2)*sin(dLat/2)
        + cos(lat1)*cos(lat2)*sin(dLon/2)*sin(dLon/2);
    double c = 2.0 * atan2(sqrt(a), sqrt(1.0 - a));
    return R * c;
}
```

### 3.2.3 Carga de los Archivos CSV

La función cargarAeropuertos lee el archivo airports\_final.csv línea por línea y construye los mapas aeropuertos e iata\_a\_id. La función cargarRutas lee routes\_final.csv y llena la lista de adyacencia, calculando la distancia geodésica para cada arista al momento de la carga. Ambas funciones usan un parseador propio que maneja correctamente los campos con comas internas encerrados en comillas.

### 3.3 Reto 1 – Alcance Personalizado

#### 3.3.1 Descripción

El usuario ingresa el nombre o el código IATA de un aeropuerto y el sistema calcula cuántos aeropuertos distintos son alcanzables realizando un máximo de 3 escalas intermedias.

#### 3.3.2 Algoritmo: BFS (Búsqueda por Amplitud)

Se utilizó BFS porque explora el grafo nivel por nivel, donde cada nivel equivale a un vuelo adicional. Esto permite controlar de manera natural la profundidad de la búsqueda.

El algoritmo mantiene en la cola pares (ID de aeropuerto, número de saltos realizados). Cuando el número de saltos llega a 3, no se exploran más vecinos desde ese nodo. Cada aeropuerto nuevo descubierto se marca como visitado para no contarlo dos veces.

```
int AlcancePersonalizado(int id_origen) {
    queue<pair<int,int>> cola; // (id_aeropuerto, saltos)
    unordered_map<int,bool> visitado;
    visitado[id_origen] = true;
    cola.push({id_origen, 0});
    int alcanzados = 0;
    while (!cola.empty()) {
        int id = cola.front().first;
        int saltos = cola.front().second;
        cola.pop();
        if (saltos >= 4) continue; // 4 saltos = 3 escalas
        for (auto& arista : adyacencia[id]) {
            int dest = arista.id_Destino;
            if (!visitado.count(dest)) {
                visitado[dest] = true;
```

```

        alcanzados++;
        cola.push({dest, saltos+1});
    }
}
}
return alcanzados;
}

```

### 3.3.3 Análisis de Complejidad

★ **Temporal:**  $O(V + E)$ , donde  $V$  es el número de aeropuertos y  $E$  el número de rutas. En el peor caso se visitan todos los nodos y aristas accesibles dentro del límite de 4 saltos.

★ **Espacial:**  $O(V)$  para la cola BFS y el mapa de visitados.

## 3.4 Reto 2 – Grupos y Aislamiento Aéreo

### 3.4.1 Descripción

Se busca identificar cuántos grupos cerrados o componentes fuertemente conexos (SCC) existen en la red, es decir, conjuntos de aeropuertos desde los cuales se puede volar hacia cualquier otro aeropuerto del mismo grupo siguiendo las rutas disponibles. También se reporta el tamaño del grupo más grande y cuántos grupos tienen más de un aeropuerto.

### 3.4.2 Algoritmo: Kosaraju

El algoritmo de Kosaraju fue elegido porque encuentra todos los componentes fuertemente conexos de un grafo dirigido en dos pasadas de DFS (búsqueda en profundidad). Su implementación se hizo de manera iterativa para evitar desbordamientos de pila en un grafo con miles de nodos.

Los tres pasos del algoritmo son:

- DFS sobre el grafo original: se recorre el grafo y se agrega cada nodo a una pila en orden de finalización (el último en terminar queda en la cima).

- Construcción del grafo transpuesto: se invierte la dirección de todas las aristas.
- DFS sobre el transpuesto: se procesa cada nodo según el orden de la pila. Cada árbol DFS en el grafo transpuesto corresponde exactamente a un SCC.

### 3.4.3 Análisis de Complejidad

- ★ **Temporal:**  $O(V + E)$ , ya que realiza exactamente dos recorridos completos del grafo (uno en el original y otro en el transpuesto).
- ★ **Espacial:**  $O(V + E)$  para almacenar el grafo transpuesto, la pila de finalización y los mapas de visitados.

## 3.5 Reto 3 – La Máxima Eficiencia (Diámetro del Grafo)

### 3.5.1 Descripción

Se busca el par de aeropuertos cuyo camino mínimo en kilómetros sea el más largo de todos los caminos mínimos posibles en el grafo. Esta medida se conoce como el diámetro del grafo y representa el "peor caso" de eficiencia en la red: los dos aeropuertos más distantes entre sí cuando se toma la ruta óptima.

### 3.5.2 Algoritmo: Dijkstra repetido

Para calcular el diámetro exacto es necesario conocer la distancia mínima desde cada nodo hacia todos los demás, lo que requiere ejecutar el algoritmo de Dijkstra una vez por cada aeropuerto de origen que tenga rutas de salida.

Dijkstra utiliza una cola de prioridad (min-heap) para explorar siempre el nodo más cercano aún no procesado. Las aristas tienen peso positivo (distancias en km), por lo que el algoritmo garantiza encontrar el camino más corto.

```
unordered_map<int,double> dijkstra(int origen) {
    unordered_map<int,double> dist;
    dist[origen] = 0.0;
    priority_queue<pair<double,int>,
                  vector<pair<double,int>>>
```

```

        greater<>> pq;
    pq.push({0.0, origen});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto& a : adyacencia[u]) {
            double nd = d + a.Distancia;
            if (!dist.count(a.id_Destino) || nd < dist[a.id_Destino]) {
                dist[a.id_Destino] = nd;
                pq.push({nd, a.id_Destino});
            }
        }
    }
    return dist;
}

```

### 3.5.3 Análisis de Complejidad

★ **Temporal:**  $O(V \times (V + E) \log V)$ , ya que se ejecuta Dijkstra una vez por cada nodo de origen ( $V$  ejecuciones), y cada ejecución de Dijkstra cuesta  $O((V + E) \log V)$ .

★ **Espacial:**  $O(V)$  por cada ejecución individual de Dijkstra.

Este reto es el más costoso computacionalmente. Para la red de 6 072 aeropuertos y 36 389 rutas, el proceso puede tardar varios minutos en ejecutarse.

## 4. Conclusiones

A partir del trabajo realizado en el modelado y análisis de la red mundial de vuelos, se pueden extraer las siguientes conclusiones:

- La limpieza de datos es un paso crítico: el dataset original presentaba un porcentaje considerable de registros inutilizables (aeropuertos sin código IATA, rutas con IDs nulos o duplicadas). Tras la limpieza se pasó de 7 697 a 6 072 aeropuertos y de 67 662 a 36 389 rutas, garantizando que el grafo refleje información real y consistente.
- La lista de adyacencia es la estructura adecuada: dado que la red es dispersa (cada aeropuerto tiene en promedio unos 6 vecinos de un total de 6 072 posibles), una lista

de adyacencia con `unordered_map` es mucho más eficiente en memoria que una matriz de adyacencia, que requeriría almacenar más de 36 millones de entradas.

- BFS es ideal para búsquedas con límite de profundidad (Reto 1): su exploración nivel por nivel permite controlar naturalmente el número de escalas, con una complejidad lineal  $O(V + E)$  que lo hace muy eficiente.
- Kosaraju revela la estructura real de conectividad (Reto 2): la identificación de componentes fuertemente conexos muestra que la red aérea mundial no es completamente conexa: existen grupos de aeropuertos desde los cuales no es posible regresar al punto de partida siguiendo las rutas disponibles.
- El cálculo del diámetro es computacionalmente costoso (Reto 3): ejecutar Dijkstra  $V$  veces es la única forma exacta de hallar el par más distante, pero su complejidad  $O(V \times (V + E) \log V)$  lo convierte en el algoritmo más exigente del proyecto..

## 5. Bibliografía

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4.<sup>a</sup> ed.). MIT Press.
- OpenFlights. (2024). Airport and airline data. Recuperado de <https://openflights.org/data.html>
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4.<sup>a</sup> ed.). Addison-Wesley.
- Polars Development Team. (2024). Polars: A lightning-fast DataFrame library. Recuperado de <https://www.pola.rs>
- cppreference.com. (2024). C++ Reference. Recuperado de <https://en.cppreference.com>